



DATA STRUCTURES

CONTENTS



22. Binary Search Trees-II

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the binary search tree ADT. In this module we will discuss some more operations of binary search tree ADT.

Learning Objectives

The learning objectives of the module are as follows:

- To illustrate the different types of Binary Search Tree Traversals
- To discuss the construction of Binary Search Trees
- To describe Insertion into Binary Search Trees
- To explain Deletion from Binary Search Trees

22.1 Introduction

As you may recall from our last module Binary Search Trees (BST) or

Lexicographic Trees are a type of Binary Trees with a special organization of data. A binary search tree is a binary tree with a special property called the BST-property. This property states that for a tree or sub-tree rooted at X, all keys to the left of X are lesser than the key at X and all keys to the right of X are greater than the key at X. We also assume that the keys of a BST are pairwise distinct.

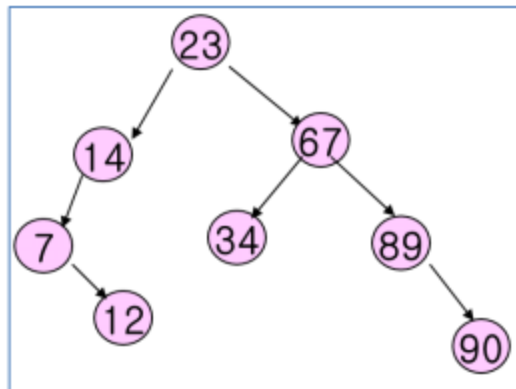


Figure 22.1 Example of a Binary Search Tree

22.2 Traversals of Binary Search Trees

Now let us consider the example of a binary search tree given in Figure 22.1. Traversals of binary search tree is similar to the traversal of binary trees but here we know that the binary tree is organized in a special way.

22.2.1 Preorder Traversal

We will first discuss the preorder traversal of a BST (Figure 22.2). Here we process the root (**23**) first. The root of the left subtree of 23 that is **14** is processed, followed by 7 root of left subtree of 14 is processed. Now since 7 has no left child, we process the right subtree of 7 that is **12**. Now we have completed processing node 7, that is processing of left subtree of 14. Now we need to process the right subtree of 14 – which in this example does not exist. Therefore we have completed processing node 14 or in other words the left subtree of the root node 23. Now we need to process the right subtree of the root 23. Now we process the root node **67**, then its

left subtree **34**. Since 34 has no children, we process the right subtree of 67 that is **89**. Now 89 has no left child so we process the right child of 89, that is **90**. Now we have completed traversing all the nodes. The preorder traversal is given in Figure 22.2.

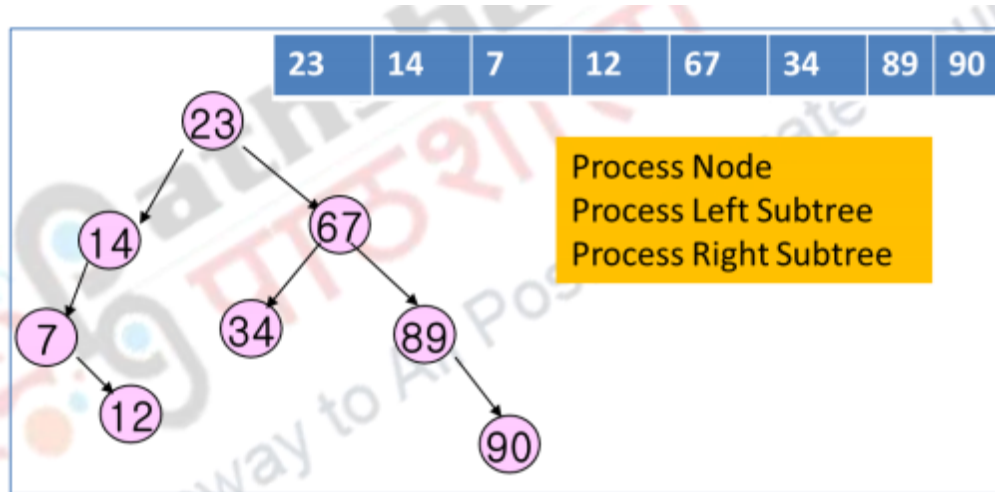


Figure 22.2 Preorder Traversal of a BST

22.2.2 Postorder Traversal

Now let us discuss the postorder traversal of the BST (Figure 22.3). Here we first need to process the left subtree of the root (23). In order to process the root of the left subtree of 23 that is subtree rooted 14 in post order, we need to process its left subtree that is subtree rooted at 7. Here since 7 has no left child, we go on to process its right subtree – here only one child **12**. Now we process node **7**. We have completed processing left subtree of node 14. Now we need to process the right subtree of 14 – which in this example does not exist. Therefore we have completed processing left and right subtree of node 14, therefore we need to process node **14**. On completion of processing node 14, we have completed processing left subtree of root node 23. Now we need to process the right subtree of the root 23 rooted at 67. Now we need to process the left subtree of 67 that is 34. Since 34 has no children, we process the node **34**. Now we need to process the right subtree of 67 rooted at 89. Now 89 has no left child so we process the right child of 89, that is **90**. Now we have completed processing left and right subtree of 89, so we process the root **89**. Now we have completed processing left and right subtree of the root 67, so we process the node **67**. Now we have completed processing left and right subtree of

the root 23, so we process the root **23**. Now we have completed processing all the nodes of the tree. The postorder traversal is given in Figure 22.3.

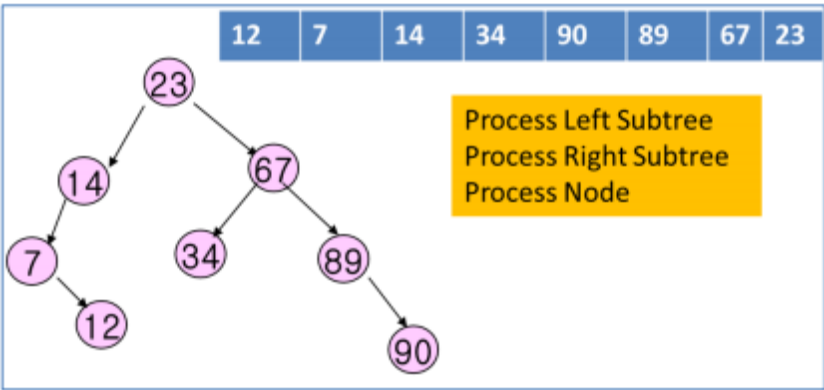


Figure 22.3 Postorder Traversal of a BST

22.2.3 Inorder Traversal

Now let us discuss the in-order traversal of the BST (Figure 22.4). Here we first need to process the left subtree of the root (23). In order to process the root of the left subtree of 23 that is subtree rooted 14 in in-order, we need to process its left subtree that is subtree rooted at 7. Here since 7 has no left child, we process the node 7. We go on to process it's right subtree – here only one child **12**. We have completed processing left subtree of node 14. Now we process node **14**. Now we need to process the right subtree of 14 – which in this example does not exist. Therefore we have completed processing left and right subtree of node 14. On completion of processing node 14, we have completed processing left subtree of root node 23. Therefore we process the node **23**. Now we need to process the right subtree of the root 23 rooted at 67. Now we need to process the left subtree of 67 that is 34. Since 34 has no left child, we process the node **34**. We proceed in the same way by processing the left subtree, then root node and then right subtree until all the nodes are processed. The in-order traversal is given in Figure 22.4. The in-order traversal of a binary search tree produces a sequence of numbers which are in increasing order.

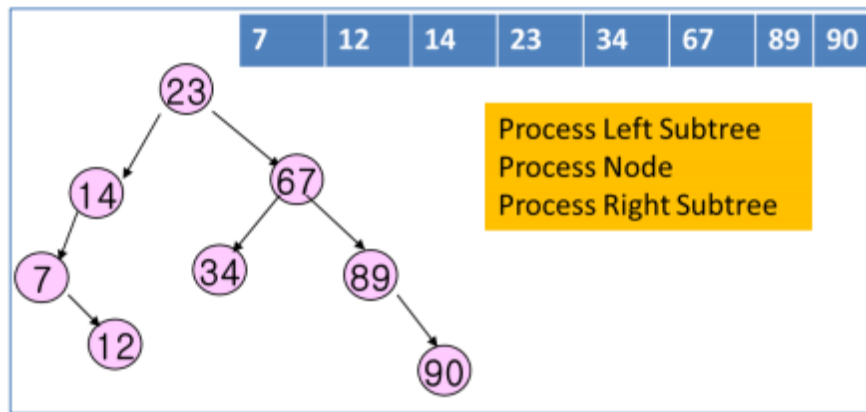


Figure 22.4 In-order Traversal of a BST

22.2.4 Right Node Left Traversal

Now for the BST another type of traversal has also been defined called the right node left traversal (Figure 22.5). This traversal is similar to inorder traversal except that instead of left subtree –root– right subtree, the traversal is carried out as right subtree –node–left subtree. This type of traversal is important for a BST because this produces a descending sequence.

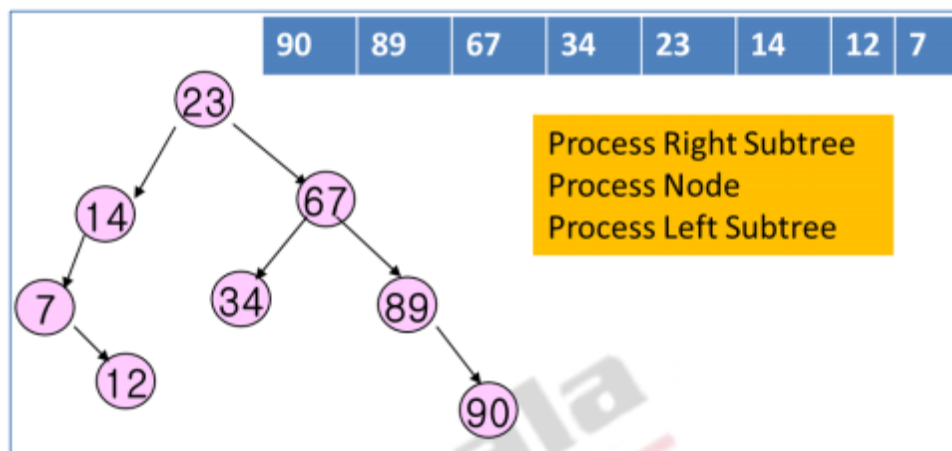


Figure 22.5 Right Node-Left Traversal of a BST

22.3 Construction of a Binary Search Tree

Now let us consider the construction of BSTs. There are two things to remember. First the tree that is constructed must obey the binary search property. Second we need to search the BST to find the correct position in which to insert a node. That is we need to maintain the key property and ensure that the smaller values are in the left sub-tree and the larger values in right sub-tree for every node in the tree. We also need to remember for the same keys, different BSTs will be constructed if the keys are used for construction in a different order.

Now the base case is when the tree is empty. In this case, we create new node for the item and make it the root of the BST. In the recursive case, if key (node to be added next) < root's value, add to the left subtree, otherwise add to the right subtree. Remember that all BST insertions take place at a leaf or a leaflike node. A leaflike node is a node that has one null subtree.

In order to insert an element e , we need to find the node n that should be e 's parent, and set n 's child to be a newly created node containing e . Now here we use the search operation in order to find the parent node n . If the element or key is smaller than the root we are at, we examine the left subtree and if the element is greater, we examine the right subtree in each case. When we can no longer proceed, we are at a leaf; this is the position at which to add the element e .

22.3.1 Algorithm

Perform search for value X

Search will end at node Y (if X not in tree). If the element to be added is a duplicate which is generally not allowed in BST we can indicate that insertion is not possible.

- If $X < Y$, we need to insert new leaf X as new left subtree of the node Y
- If $X > Y$, we need to insert new leaf X as new right subtree of the node Y

22.3.2 Observations

The search operation will be $O(\log(n))$ only for balanced trees. We have defined balanced trees as trees where for each node the height of the left and right subtrees differ by at most one. Now insertions into BSTs may unbalance the tree. In fact

when we insert the keys in ascending or descending order we will land up with skewed trees. Insertion always places a new item near the periphery (that is at the bottom of the tree) of the BST while ensuring that the BST property is maintained. Searching of the binary search tree is used to locate the insertion point. As in a search operation, starting at the root we probe down the tree till we find a node who's left or right sub-tree is empty. Now we have found the place for the new value. The insertion is based on comparisons of the new item and values of nodes in the BST. Please note that elements in nodes must be comparable otherwise we cannot construct a BST.

22.3.3 Detailed Cases

Case 1: Let us first consider the case when the tree is empty

- Here we set the root to a new node containing the item – ***create a root node with the new key***

Case 2: Now let us assume that the tree is not empty *Here we compare key with the top node*

```
if key = node key
The node already exists
else if key > node key
compare key with the right subtree:
if subtree is empty create a leaf node
else add key in right subtree
else key < node key
compare key with the left subtree:
if the subtree is empty create a leaf node
else add key to the left subtree
```

The steps here include the search procedure to find the correct location for insertion as well as the actual insertion.

The trace of the recursive BST insert algorithm is given in Figure 22.6. Now let us assume that we want to insert node 19 into the existing BST rooted at 23. Now we find $19 < 23$, so we go left, where we find $19 > 14$ so we go right. Now we find that the

right subtree of 14 is empty so we insert there. The BST after insertion is shown in Figure 22.6.

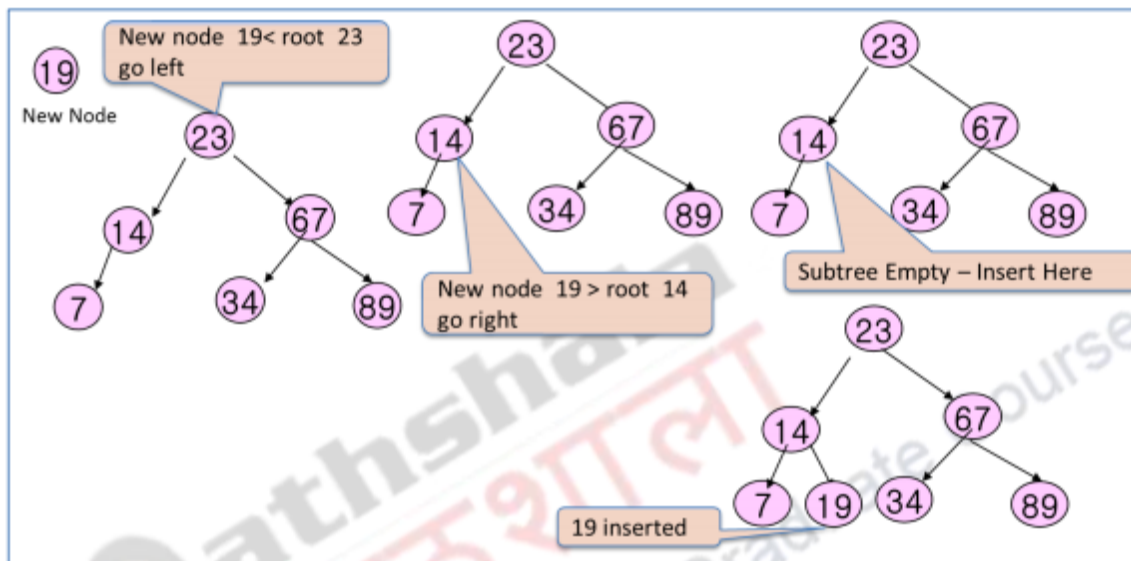


Figure 22.6 Trace of recursive BST Insert

22.3.4 Construction/Insertion of a Binary Search Tree – Example

Add the elements 23, 67, 14, 89, 14, 7, 90, 12 to form a Binary Search Tree (Figure 22.7).

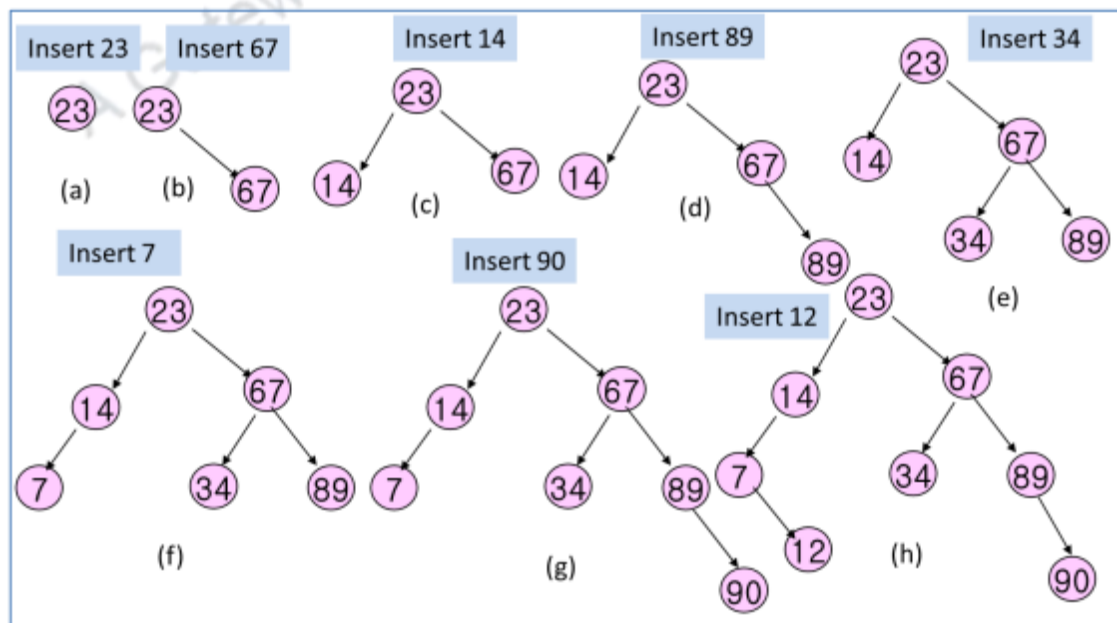


Figure 22.7 Example of Insertion of Keys into BST

1. Initially BST is Empty-create a root node with the new key 23 (Figure 22.7 (a)).
2. Now we need to insert 67, which is >23 , we have an empty right subtree so we directly insert it as right subtree of 23 (Figure 22.7 (b)).
3. Next we insert node 14, which is <23 , we have an empty left subtree so we directly insert it as left subtree of 23 (Figure 22.7 (c)).
4. Next we insert node 89, $89 > 23$, so we go right, but right subtree not empty, so we check 89 with root of right subtree 67. We find $89 > 67$, so we go right. Now we find an empty right subtree and insert 89 there (Figure 22.7 (d)).
5. Next we need to insert node 34. $34 > 23$, so we go right, but right subtree not empty, so we check 34 with root of right subtree 67. We find $34 < 67$, so we go left. Now we find an empty left subtree and insert 34 there (Figure 22.7 (e)).
6. Next we need to insert node 7. $7 < 23$, so we go left, but left subtree not empty, so we check 7 with root of left subtree 14. We find $7 < 14$, so we go left. Now we find an empty left subtree and insert 7 there (Figure 22.7 (f)).
7. Next we need to insert node 90. $90 > 23$, so we go right, but right subtree not empty, so we check 90 with root of right subtree 67. We find $90 > 67$, so we go right. But right subtree not empty, so we check 90 with root of right subtree 89. We find $90 > 89$, so we go right. Now we find an empty right subtree and insert 90 there (Figure 22.7 (g)).
8. Next we need to insert node 12. $12 < 23$, so we go left, but left subtree not empty,

so we check 12 with root of left subtree 14. We find $12 < 14$, so we go left. But left subtree not empty, so we check 12 with root of left subtree 7. We find $12 > 7$, so we go right. Now we find an empty right subtree and insert 12 there (Figure 22.7 (h)).

22.3.5 Analysis

The time taken for initialization will be of the $O(1)$. The while loop searches for place to insert *the key*, while maintaining the parent. This $O(h)$ time where h is the height of the tree. The actual creation of the node for insertion of the value takes $O(1)$. Therefore total time taken for insertion of a node will be of $O(h)$.

22.4 Removing or Deleting from Binary Search Tree

When removing a node from a BST, we need to ensure that the binary search tree property is maintained. Therefore during deletion, we need to search for the node, remove it, and do some reorganization to maintain the binary search tree property. There are many cases two be considered, we may be removing an element which is a leaf, removing an element with one child, removing an element having both the children or we may be deleting the root itself. Again we can have both recursive and an iterative implementation. If the node being deleted is a leaf then deletion is a straightforward procedure of just making the associated parent node point to NULL. If the node being deleted has only one child then the parent node of the node being deleted will have to point to the child node of the node being deleted. If the node being deleted has both the children then we first need to find the **inorder successor** of the node being deleted and replace the node being deleted with this node. (The inorder successor of a node can be obtained by taking the right node of the current node and traversing in the left till we reach the left most node whose right subtree is null) We can also take inorder predecessor instead of inorder successor. The total time taken for deletion is of the $O(h)$.

The algorithm removes a specified item from the BST and adjusts the tree. It uses a binary search to locate the target item that is starting at the root it probes down the tree till it finds the target or reaches a leaf node (target not in the tree). The removal of a node must not leave a 'gap' in the tree.

22.4.1 Algorithm

1. if the tree is empty return false
2. Attempt to locate the node x using the binary search algorithm
if the node is not found return false else the node is found, remove it
3. **(a) Case 1:** if x has no children then remove x (The node to be deleted has no children. In this case, all we need to do is delete the node)
if the node has 2 empty subtrees
replace the link in the parent of x which was pointing to x with null
(b) Case 2: if x has one child (left) then make $parent[x]$ point to existing child of x . The node to be deleted has only a left subtree. We delete the node and attach the left subtree to the deleted node's parent.
if the node has no right child (only left child)
link the parent of the target to the left (non-empty) subtree
(c) Case 3: if x has one child (right) then make $parent[x]$ point to existing child of x . The node to be deleted has only a right subtree. We delete the node and attach the right subtree to the deleted node's parent.
if the node has no left child (only right child)
link the parent of the node to the right (non-empty) subtree
(d) Case 4: if x has two children (subtrees) then swap x with its in order successor – perform case 0 or case 1 to delete it. The node to be deleted has two subtrees. Rather than simply delete the node, we try to maintain the existing structure as much as possible by finding data to take the place of the deleted data. This can be done in one of two ways. We can find the largest node in the deleted node's left subtree and move its data to replace the deleted node's data. We can find the smallest node on the deleted node's right subtree and move its data to replace the deleted node's data. Either of these moves preserves the binary search property of the tree. Moreover the property of inorder successor (or predecessor) ensures that this node will have either no child or at most one child. The inorder successor (respectively, the predecessor) of a key k in a search tree is the smallest (respectively, the largest) key that belongs to the tree and that is strictly greater than (respectively, less than) k . The idea for finding the successor of a given node x is that if x has right child, then the successor is the minimum in the right subtree of x otherwise, the successor is the parent of the farthest node that can be reached from x by following only right branches backward. The predecessor can be found similarly with the roles of left and right exchanged and with the roles of maximum and minimum exchanged.

Before we go further let us describe the algorithm to find the inorder successor

of a node x .

The successor (x) = y , is such that key [y] is the smallest key $>$ key [x]. Here there are two cases

- Case 1: right (x) is non empty
 - successor (x) = the minimum in right (x)
- Case 2: right (x) is empty
 - go up the tree until the current node is a left child: successor (x) is the parent of the current node
 - if you cannot go further (and you reached the root): x is the largest element

The algorithm is given in Figure 22.8.

```

TREE-SUCCESSOR( $x$ )
  if right [ $x$ ]  $\neq$  NIL
    then return TREE-MINIMUM(right [ $x$ ])
   $y \leftarrow p[x]$  – (parent of  $x$ )
  while  $y \neq$  NIL and  $x =$  right [ $y$ ]
    do  $x \leftarrow y$ 
     $y \leftarrow p[y]$ 
  return  $y$ 

```

Figure 22.8 In order Successor Algorithm

if the node has a left and a right subtree

- **replace the node's value with the minimum value in the right subtree (inorder successor)**
- **delete the minimum node in the right subtree**

Instead of using the inorder successor as we described above, we can use the inorder predecessor defined in a similar way.

We have explained the concept of the deletion algorithm and detailed the four cases we consider during deletion. We use the Tree Successor algorithm to find the inorder successor when the node to be deleted has both left and right children. We assume that the parent of a node is available. Now let us assume that we want to delete the node z from the binary search tree. If z has no children, then we will just

replace z by nil. If z has only one child, then we will promote the unique child to z's place. If z has two children, then we will identify z's successor. Call it y. The successor y either is a leaf or has only the right child. We promote y to z's place. Now the deletion of y from its original place will result in deletion of the first three cases only. The code for the above is given in Figure 22.9.

```

BST-Delete(T, z) /*Here we are deleting node z from the tree T*/
/* Case 1, Case2 & Case3*/
if left[z] = nil or right[z] = nil
    then y ← z
    else y ← BST-Successor(z) - /* y is the node that's actually removed */
/*Here y does not have two children*/
if left[y] ≠ nil
    then x ← left[y]
    else x ← right[y] - /* x is the node that is moving to y's position */
if x ≠ nil then p[x] ← p[y] /* parent of x is reset if x isn't NIL. Resetting is
                           unnecessary if x is NIL*/
if p[y] = nil
    then root[T] ← x /* If y is the root, then x becomes the root*/
    /*Otherwise, do the following*/
    else if y = left[p[y]] /*If y is the left child of its parent*/
        then left[p[y]] ← x /*Set the parent's left child to x*/
        else right[p[y]] ← x /*If y is the right child of its parent, then Set the
                             parent's right child to x*/
if y ≠ z then
    { key[z] ← key[y]
      Move other data from y to z }
return (y)

```

Figure 22.9 Algorithm for Deletion

22.4.2 Deletion from BST – Example

Now let us discuss each of the 4 cases of deletion with examples. Let us consider Case 1 where the node to be deleted is a leaf that is has no children. Let us consider deletion of node 34 (Figure 22.10). We need to carry out the search operation (as was done for insertion) to find 34. Now parent of 34 that is 67 would have 34 as its left child. The parent 67's left child is now made Null.

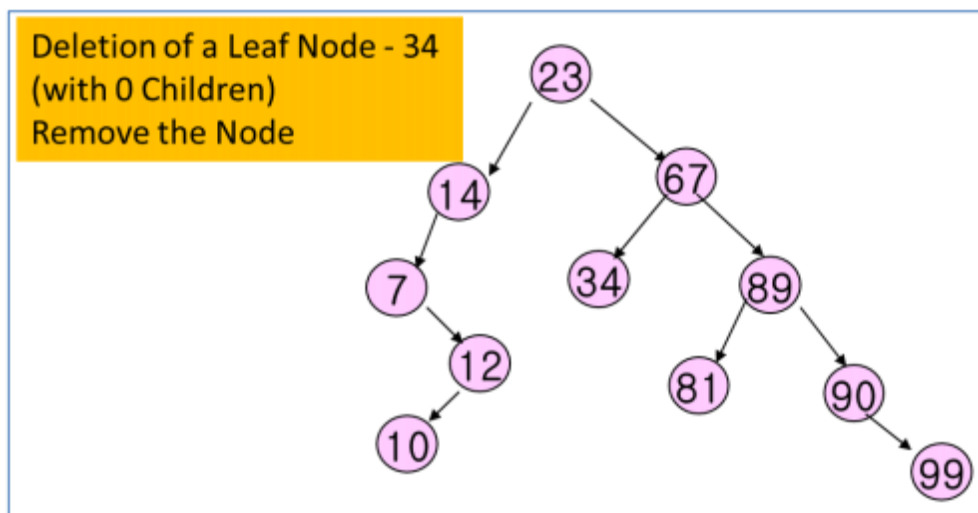


Figure 22.10 Deletion from BST (Case (1))

Now let us consider Case 2 where the node to be deleted has one child, in this only left child. Let us consider deletion of node 12 (Figure 22.11). We need to carry out the search operation (as was done for insertion) to find 12. Now 12 has only left child that is 10. Node 12 is deleted and 12's parent 7's right child is now the left child of 12 that is 10.

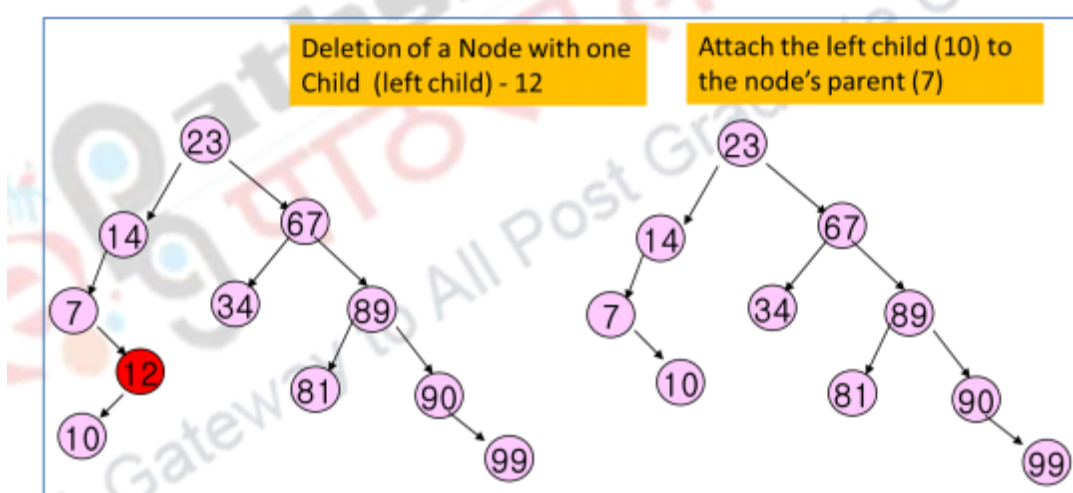


Figure 22.11 Deletion from BST (Case (2))

Now let us consider Case 3 where the node to be deleted has one child, in this only right child. Let us consider deletion of node 90 (Figure 22.12). We need to carry

out the search operation (as was done for insertion) to find 90. Now 90 has only a right child that is 99. Node 90 is deleted and 90's parent 89's right child is now the right child of 90 that is 99.

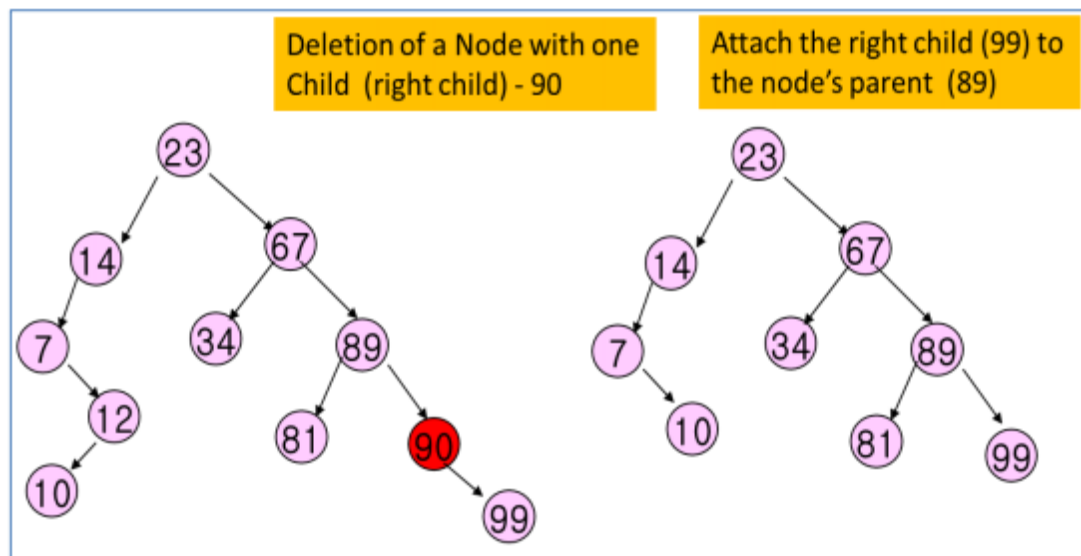


Figure 22.12 Deletion from BST (Case (3))

Now let us consider Case 4 where the node to be deleted has both the children. Let us consider deletion of node 67 (Figure 22.13). We need to carry out the search operation (as was done for insertion) to find 67. Now 67 has both left (34) and right (89) children. First we need to find the inorder successor of 67, which is 81. Now we replace 67 in tree with its inorder successor 81. In order to do this we first need to delete 81 from its place in the tree. By definition of inorder successor, 81 can have at most one child. In our case it has no children. Therefore to delete 81 we make the left child of its parent (89) as Null. To replace 67 with 81, the parent of 67 that is 23's right child should now point to 81. Moreover 81's left pointer should be made to point to 34 (67's left child) and 81's right pointer should be made to point to 89 (67's right child). This completes the deletion process.

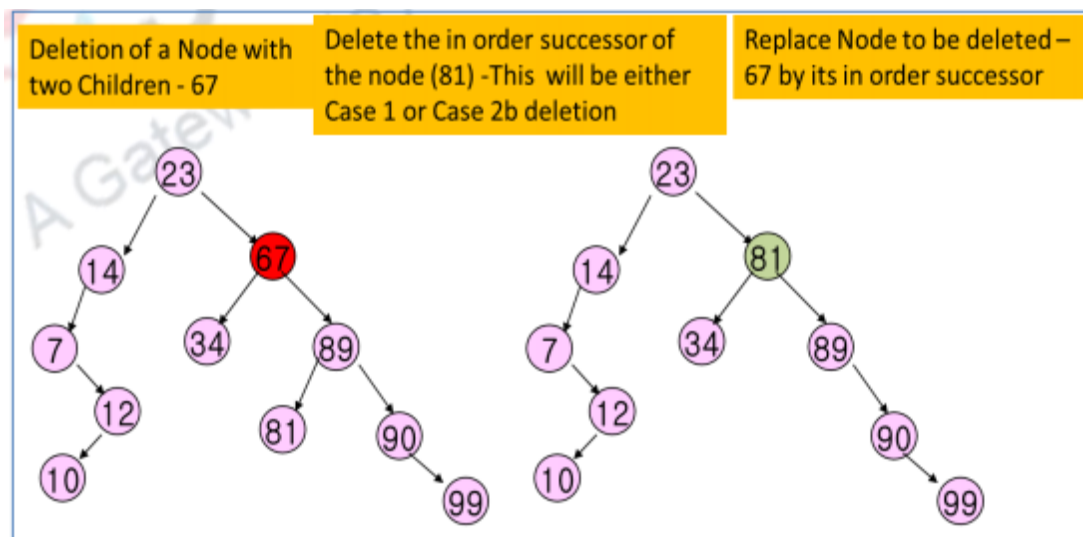


Figure 22.13 Deletion from BST (Case (4))

Summary

- Illustrated the different type of Binary Search Tree Traversals with Examples
- Discussed the Construction of Binary Search Trees
- Described the Insertion operation of Binary Search Trees
- Explained the Deletion Operation of Binary Search Trees

you can view video on Binary Search Trees-II



Web Links
https://www.cs.princeton.edu/~rs/AlgsDS07/08BinarySearchTrees.pdf
http://algs4.cs.princeton.edu/32bst/
http://www.cs.swarthmore.edu/~newhall/unixhelp/Java_bst.pdf
https://www.cs.rochester.edu/~gildea/csc282/slides/C12-bst.pdf
https://www.cs.uic.edu/pub/CS202/LectureNotes/10trees.pdf
imranihsan.com/data/2013/dsa/DSAF1312.pdf
https://www.cs.rochester.edu/~gildea/csc
https://www.cs.umd.edu/class/summer2015/
http://www.eagle.tamut.edu/faculty/igor/

Supporting & Reference Materials
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Robert Sedgewick, Kevin Wayne "Algorithms", Addison-Wesley Professional, 4th Edition, 2011
Thomas H. Cormen, et.al, "Introduction to Algorithms" The MIT Press; 3rd edition, 2009
Anany Levitin "Introduction to the Design and Analysis of Algorithms", Pearson; 3rd edition 2011